

TRISHA

Text Recognition and Intelligent Speech Helper Assistant

Document Type	System Architecture & Project Planning Documentation
Project	A Flutter-Based Accessibility Application for Individuals with Visual Impairments
Institution	National University Philippines
Program	Bachelor of Science in Information Technology
Document Version	1.0 — Capstone Submission

Table of Contents

1.	Project Overview & Objectives
2.	System Architecture
2.	
1	Architecture Overview Diagram
2.	
2	Component Descriptions
2.	
3	Deployment Architecture
3.	Technology Stack
4.	Module Design & API Interfaces
5.	Data Flow Diagrams
6.	Database Design (MongoDB Schemas)
7.	Project Roadmap & Milestones
8.	Cost Breakdown
9.	Security Architecture
10.	Scalability & Performance Planning
11.	React Web — Information Site & Superadmin Panel
12.	Technical Notes & Annotations

1. Project Overview & Objectives

Project Overview

TRISHA — National University Philippines Capstone

TRISHA (Text Recognition and Intelligent Speech Helper Assistant) is a Flutter-based mobile application designed to empower individuals with visual impairments in the Philippines through accessible, AI-enhanced text recognition and speech synthesis. The system combines on-device OCR via Google ML Kit, multi-language TTS including Filipino/Taglish, and an AI-powered contextual explanation engine backed by a full MERN-stack infrastructure.

Core Objectives

- Provide offline-capable OCR and TTS for visually impaired users without requiring constant internet access.
- Enable AI-powered contextual explanation of scanned text using OpenAI GPT-4o-mini or Google Gemini.
- Support Filipino language TTS output, addressing a gap in existing foreign accessibility apps.
- Deliver a gesture-based, voice-command-navigable mobile interface requiring no visual interaction.
- Provide an administrative web portal for user management, audit logging, and feedback review.
- Build a public-facing information website (GCash-style) for app downloads and feature documentation.

Novelty Statement

Existing solutions (Microsoft Seeing AI, Google Lookout) are foreign products with no Filipino language TTS, no local offline support, and no AI explanation layer. TRISHA's novelty: (1) hybrid on-device + cloud OCR pipeline, (2) AI contextual Q&A; on scanned text, (3) native Filipino/Taglish TTS, (4) full voice-command navigation, and (5) a MERN-stack multi-tenant admin system designed for Philippine institutional deployment.

Target Client

Proposed institutional client: National Council on Disability Affairs (NCDA), Deaf and Blind School of the Philippines (Manila), or Tahanang Walang Hagdanan — any of which qualifies as a formal external client for NU panel review.

2. System Architecture

2.1 Architecture Overview Diagram

Four-tier: Flutter → Express → MongoDB | React Admin

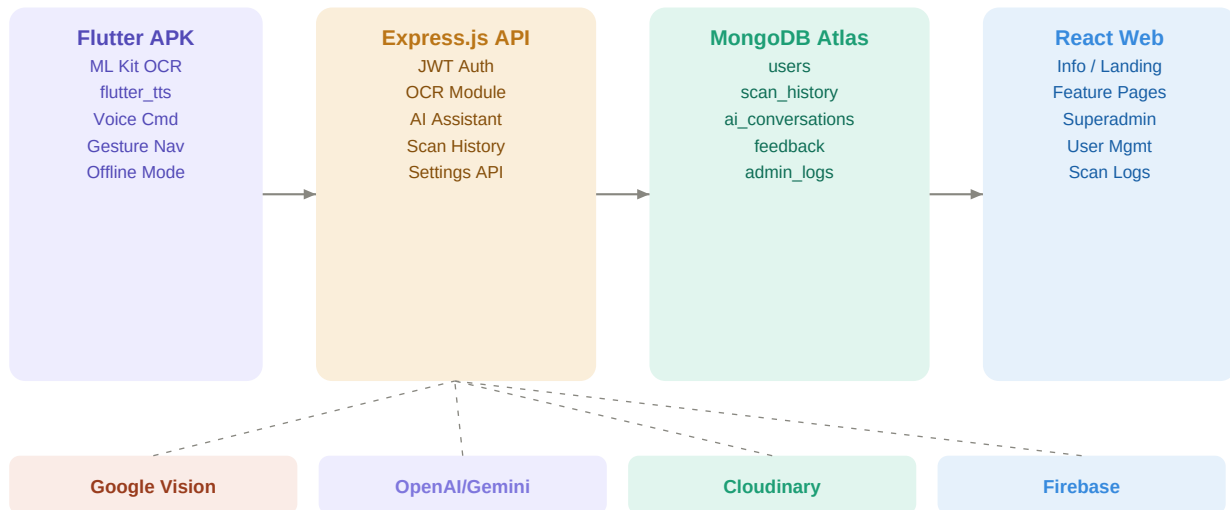


Figure 1 — TRISHA system architecture: Flutter mobile client, Express.js REST API, MongoDB Atlas database, React web portal, and external service integrations.

2.2 Component Descriptions

Flutter APK (Mobile Client)

- Handles camera capture, gallery image selection, and image pre-processing before OCR.
- Runs Google ML Kit OCR entirely on-device — no internet required for core text extraction.
- Uses flutter_tts with Filipino language pack for text-to-speech output.
- Implements speech_to_text for full voice command navigation (read, explain, save, repeat).
- Gesture-based UI with large tap targets and swipe navigation — no visual interaction required.
- Syncs scan history and user settings with backend when online; queues requests offline.

Express.js REST API (Backend)

- Stateless REST API built on Node.js 20 LTS and Express 4.x with modular route structure.
- JWT-based authentication with bcrypt password hashing (rounds=12).
- OCR fallback module using Google Vision API for complex or low-quality document scans.
- AI Assistant module proxying to OpenAI GPT-4o-mini or Gemini 1.5 Flash with conversation context.
- Cloudinary integration for signed image uploads with transformation and CDN delivery.
- Role-based access control: superadmin, org_admin, and user roles enforced per route.

MongoDB Atlas (Database)

- M0 free shared cluster sufficient for capstone demo and initial production usage.
- Mongoose ODM for schema validation, virtuals, and pre/post hooks.
- Collections: users, scan_history, ai_conversations, feedback, admin_logs, organizations.
- Indexes on userId, organizationId, createdAt for query performance.
- Atlas provides built-in backups, monitoring, and IP allowlisting for security.

React Web Portal (Frontend)

- Public information/landing site in GCash.com style: hero, features, about, APK download.
- Superadmin dashboard with user management, tenant/org management, and system logs.
- Built with React 18 + Vite 5 + TailwindCSS 3 for fast build and deployment.
- Protected admin routes with JWT token refresh and role-based route guards.
- Deployed on Vercel free tier with CDN edge delivery.

2.3 Deployment Architecture

Flutter APK is distributed via direct APK sideload (for capstone demo) and optionally via Google Play (\$25 one-time developer account). The Express.js backend deploys to Render.com free tier (750 hours/month), with MongoDB Atlas M0 as the managed database. The React web portal deploys to Vercel with automatic GitHub-triggered deploys. Cloudinary handles all image storage and CDN delivery. No VPS or server provisioning is required for the capstone phase.

3. Technology Stack

Full Technology Stack

Mobile · Backend · Database · Web · DevOps

Layer	Technology	Version / Notes
Mobile	Flutter + Dart	Flutter 3.x, Dart 3.x
OCR (local)	Google ML Kit	google_ml_kit ^0.16
TTS	flutter_tts	flutter_tts ^3.8 — Filipino support
Voice input	speech_to_text	speech_to_text ^6.3
Backend	Node.js + Express.js	Node 20 LTS, Express 4.x
Database	MongoDB + Mongoose	MongoDB Atlas M0, Mongoose 8.x
Auth	JWT + bcrypt	jsonwebtoken, bcrypt ^5
Cloud OCR	Google Vision API	Fallback for complex docs
AI	OpenAI GPT-4o-mini	or Gemini 1.5 Flash (free tier)
Image store	Cloudinary	Free 25GB, signed uploads
Frontend	React + Vite	React 18, Vite 5, TailwindCSS 3
Deployment	Render.com + Vercel	Backend: Render; Web: Vercel
CI/CD	GitHub Actions	Lint + test on PR merge

All dependencies use LTS or stable release versions as of the project start date. Free-tier services are selected to eliminate monthly costs during development and panel defense.

4. Module Design & API Interfaces

REST API Endpoints

Express.js · JWT Protected · Role-Based Access

Module	Endpoint / Interface	Description
Auth	POST /api/auth/register	Register user, hash password, return JWT
Auth	POST /api/auth/login	Validate credentials, return JWT
Scan	POST /api/scan/upload	Upload image, run OCR, save to history
Scan	GET /api/scan/history	Fetch user's scan history (paginated)
Scan	DELETE /api/scan/:id	Delete a scan entry
AI	POST /api/ai/explain	Send OCR text to GPT/Gemini, return response
AI	GET /api/ai/history	Fetch AI conversation history
User	GET /api/user/profile	Get user profile and preferences
User	PATCH /api/user/settings	Update TTS speed, language, font size
Admin	GET /api/admin/users	Superadmin — list all users + orgs
Admin	PATCH /api/admin/user/:id	Superadmin — update user role/status
Admin	GET /api/admin/logs	System audit logs
Feedback	POST /api/feedback	Submit OCR quality report
Feedback	GET /api/feedback	Admin — view all feedback

Express.js Folder Structure

```
src/  
modules/  
  auth/ route, controller, service, middleware  
  scan/ route, controller, service (OCR + Cloudinary)  
  ai/ route, controller, service (OpenAI/Gemini proxy)  
  user/ route, controller, service  
  admin/ route, controller, service (superadmin only)  
  feedback/ route, controller, service  
models/ Mongoose schemas  
middleware/ auth guard, role guard, rate limiter  
config/ db.js, cloudinary.js, openai.js  
utils/ response helpers, logger  
app.js Express app setup  
server.js HTTP server entry point
```

5. Data Flow Diagrams

Primary Data Flow — OCR to Speech

Capture → Extract → Store → Explain → Speak

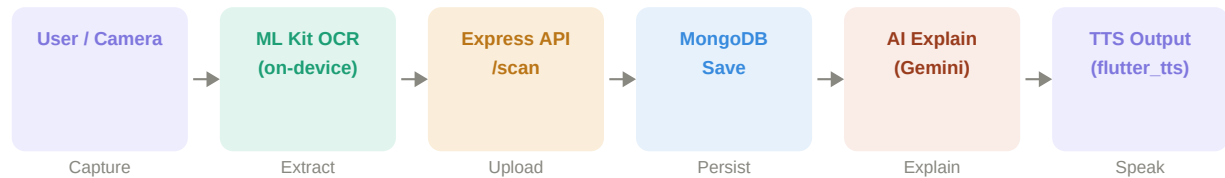


Figure 2 — Primary data flow: image capture through ML Kit OCR, API upload, MongoDB persistence, AI explanation, and TTS output.

Offline Flow (No Internet)

- User opens TRISHA app — camera launches immediately via gesture or voice.
- Image captured → Google ML Kit OCR runs entirely on-device (no API call).
- Extracted text is read aloud via flutter_tts using the device's TTS engine.
- Scan is queued locally in SQLite/SharedPreferences with pending sync status.
- When internet is restored, queued scans are uploaded to Express API and saved to MongoDB.

AI Explanation Flow

- User says 'Explain this' after scan — flutter_tts pauses, speech_to_text listens.
- App sends OCR text + user question to POST /api/ai/explain.
- Express AI module forwards to OpenAI GPT-4o-mini with system prompt for accessibility context.
- Response is returned to Flutter, stored in ai_conversations collection.
- TTS reads the AI response aloud; user can ask follow-up questions conversationally.

6. Database Design — MongoDB Schemas

MongoDB Collections

Atlas M0 · Mongoose ODM · 5 Core Collections

users <pre>_id: ObjectId name: String (required) email: String (unique, required) password: String (hashed) role: String (user org_admin superadmin) organizationId: ObjectId (ref: organizations) settings: { ttsSpeed: Number, language: String, fontSize: Number } createdAt: Date updatedAt: Date</pre>	scan_history <pre>_id: ObjectId userId: ObjectId (ref: users) imageUrl: String (Cloudinary URL) extractedText: String ocrEngine: String (mlkit vision) language: String isSynced: Boolean createdAt: Date</pre>
ai_conversations <pre>_id: ObjectId userId: ObjectId (ref: users) scanId: ObjectId (ref: scan_history) messages: [{ role: String, content: String, timestamp: Date }] model: String (gpt-4o-mini gemini-1.5-flash) createdAt: Date</pre>	organizations <pre>_id: ObjectId name: String (required) type: String (school ngo government other) contactEmail: String adminId: ObjectId (ref: users) isActive: Boolean createdAt: Date</pre>
feedback <pre>_id: ObjectId userId: ObjectId (ref: users) scanId: ObjectId (ref: scan_history) rating: Number (1-5) issue: String (wrong_text poor_tts app_crash other) notes: String status: String (open reviewed resolved) createdAt: Date</pre>	

7. Project Roadmap & Milestones

20-Week Development Roadmap

6 Phases · 3–4 Team Members · Parallel Workstreams

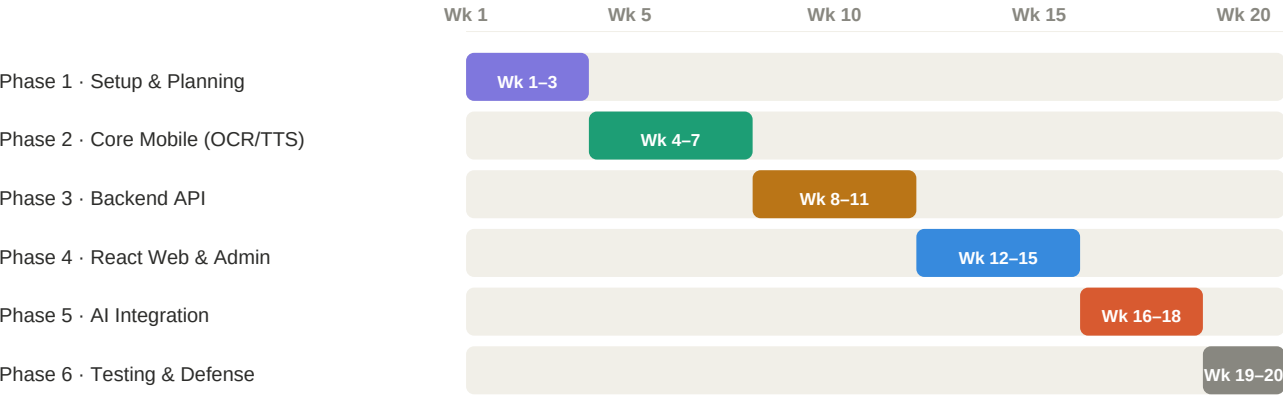


Figure 3 — 20-week Gantt chart: parallel Flutter (mobile) and Express (backend) development from Week 4, React web from Week 12.

Phase 1 — Project Setup & Planning

Weeks 1–3

- Initialize GitHub monorepo with branch protection and PR templates.
- Scaffold Flutter project, Express.js backend, and React Vite app.
- Set up MongoDB Atlas M0 cluster and connect via Mongoose.
- Configure Cloudinary, environment variables, and .env.example files.
- Write Chapter 1 (Introduction, Background, Statement of the Problem).
- Define wireframes for Flutter screens and React admin layout.

Phase 2 — Core Mobile Features

Weeks 4–7

- Integrate google_ml_kit for on-device OCR with camera and gallery input.
- Implement flutter_tts with Filipino and English language switching.
- Build voice command system using speech_to_text package.
- Design gesture navigation: swipe, long-press, and large tap zone UI.
- Implement offline scan queue with local storage and sync indicator.
- Unit test OCR accuracy across printed text, handwriting, and signage.

Phase 3 — Backend API Development

Weeks 8–11

- Build all Express route modules: auth, scan, ai, user, admin, feedback.
- Implement JWT middleware, role guard, and request validation (Joi/Zod).
- Integrate Google Vision API as OCR fallback endpoint.
- Build Cloudinary upload service with signed URL generation.
- Set up express-rate-limit and CORS policy for mobile client.
- Write Postman collection and API documentation.

Phase 4 — React Web Portal	Weeks 12–15
■ Build public information site: hero section, features, testimonials, APK download. ■ Implement superadmin authentication with JWT and protected route guards. ■ Build user management table: search, filter, role assignment, deactivation. ■ Build organization/tenant management for multi-tenant access control. ■ Build scan logs viewer, AI conversation logs, and feedback management. ■ Deploy to Vercel and connect to live Express API.	
Phase 5 — AI Integration & System Sync	Weeks 16–18
■ Integrate OpenAI GPT-4o-mini (or Gemini 1.5 Flash) via Express AI module. ■ Connect Flutter AI chat UI to backend — real-time response streaming. ■ Full end-to-end integration test: Flutter → Express → MongoDB → React. ■ Performance audit: API response times, OCR latency, TTS startup time. ■ Accessibility audit: screen reader compatibility, contrast ratios, font sizes.	
Phase 6 — Testing, Polish & Defense	Weeks 19–20
■ Final bug fixes based on UAT with visually impaired test users (if accessible). ■ APK release build — sign and sideload on test devices. ■ Finalize Chapters 3–5: methodology, results, conclusions, recommendations. ■ Prepare demo script: camera scan → TTS → AI explain → admin review flow. ■ Panel defense rehearsal with technical Q&A; preparation.	

8. Cost Breakdown

Infrastructure Cost Analysis

Development phase · Capstone demo · No monthly fees required

Service	Usage Tier	Monthly Cost	Status
MongoDB Atlas	M0 — 512MB shared	₱0	Free
Render.com (API)	Free tier — 750hr/mo	₱0	Free
Vercel (React web)	Hobby — unlimited deploys	₱0	Free
Cloudinary	25 credits/mo, ~25k transforms	₱0	Free
Google ML Kit OCR	On-device — bundled	₱0	Free
flutter_tts	Device TTS engine	₱0	Free
Google Vision API	1,000 req/mo free quota	₱0	Free
OpenAI GPT-4o-mini	~500 queries/mo (demo)	~₱600	Paid
Domain (.com.ph)	Optional — 1 year	₱600/yr	Optional
Google Play	One-time dev account	₱1,400	One-time
	Minimum monthly total	₱0	
	Maximum monthly (with AI)	~₱1,200	

The entire TRISHA stack runs at zero monthly cost during development and panel defense. OpenAI GPT-4o-mini is the only variable cost (~₱600/month for ~500 AI queries) and can be replaced with Google Gemini's free tier (60 req/min) to eliminate all operational expenses. The one-time costs (domain: ₱600/yr, Google Play: ₱1,400) are optional for the capstone submission and only required for public release.

9. Security Architecture

Security Layers

Auth · Transport · Validation · Storage · RBAC

Layer	Mechanism	Tool / Method
Authentication	JWT (HS256, 1h expiry)	jsonwebtoken
Password storage	Hashed with salt	bcrypt (rounds=12)
Transport	TLS/HTTPS everywhere	Let's Encrypt / Render
API protection	Rate limiting + CORS	express-rate-limit, cors
Input validation	Schema validation	Joi / Zod
Image storage	Signed URLs, private bucket	Cloudinary signed upload
Role-based access	Middleware guard per route	Custom RBAC middleware
MongoDB	Atlas IP allowlist + auth	Connection string with auth
Mobile	Certificate pinning (optional)	Flutter http overrides

RBAC Role Matrix

Permission	User	Org Admin	Superadmin
Scan & OCR	✓	✓	✓
View own history	✓	✓	✓
Manage own settings	✓	✓	✓
View org users	✗	✓	✓
Assign org roles	✗	✓	✓
View all organizations	✗	✗	✓
Manage all users	✗	✗	✓
View system audit logs	✗	✗	✓
Delete any scan/feedback	✗	✗	✓

10. Scalability & Performance Planning

Scalability Strategy

Stateless API · Atlas scaling · CDN delivery · Caching

Backend Scalability

- Express.js API is stateless (JWT, no server-side sessions) — horizontally scalable without configuration changes.
- Render.com supports auto-scaling to multiple instances; upgrade from free to Starter (\$7/mo) when needed.
- Rate limiting (express-rate-limit) prevents abuse; can migrate to Redis-backed rate limiting at scale.
- MongoDB Atlas scales from M0 (free) → M10 → M30 with zero code changes — just a cluster tier upgrade.

Performance Targets

- OCR latency (on-device ML Kit): target <2 seconds for standard A4 document at 12MP camera resolution.
- API response time: target <500ms for scan upload (excluding Cloudinary transfer); <1s for AI explanation.
- TTS startup: target <300ms from text-ready to first spoken word using flutter_tts warm-up on app launch.
- React web: Lighthouse score target >90 (performance, accessibility, SEO) — static deploy on Vercel CDN.

Caching Strategy

- Flutter caches last 10 scan results locally using Hive or SharedPreferences for instant re-read.
- Express can add node-cache or Redis for AI response caching — identical text queries reuse cached results.
- Cloudinary CDN caches image delivery globally; signed URLs expire after 1 hour for security.
- React web is statically generated — Vercel edge caches all pages globally with <50ms TTFB worldwide.

Future Scaling Considerations

- Message queue (Bull/BullMQ + Redis) for async OCR job processing when load increases beyond 100 concurrent users.
- WebSocket or SSE for real-time AI response streaming to Flutter client (replaces polling).
- Separate microservice for AI module to allow independent scaling and model provider switching.
- MongoDB Atlas Search for full-text search across scan history at scale.

11. React Web — Information Site & Superadmin Panel

React Web Portal

Public info site (GCash-style) + Protected superadmin dashboard

11.1 Public Information Site (GCash-style)

The public site functions as the project's marketing and distribution hub — modeled after GCash.com with a hero section, feature highlights, social proof, and a prominent APK download button. It establishes credibility for the panel and serves as the client organization's resource hub.

- Hero — app name, tagline ('See the world through sound'), and Download APK CTA button.
- Features — icon grid: Point & Read, AI Explain, Filipino TTS, Voice Control, Offline Mode, Scan History.
- How It Works — 3-step illustrated flow: Capture → Extract → Listen.
- About — project background, team members, National University branding, client organization logo.
- Contact / Support — feedback form (POST /api/feedback), FAQ accordion.
- Download — APK direct download link, system requirements, installation guide for sideloading.

11.2 Superadmin Panel

The superadmin panel is a protected React SPA accessible only to users with the superadmin role. It provides full visibility and control over the TRISHA platform — users, organizations (tenants), scan logs, AI usage, and system health.

Page	Key Features
Dashboard	Total users, scans today, AI queries, active orgs — metric cards + chart
User Management	Search/filter all users, assign roles, deactivate, view scan count
Organization / Tenancy	Create/edit orgs, assign org_admin, view member count and usage
Scan Logs	View all scans — filter by user, org, date, OCR engine; preview image
AI Conversations	View all AI chat sessions, token usage per user, model used
Feedback	Table of user reports — filter by status, assign reviewer, mark resolved
Audit Logs	System-level actions: logins, role changes, deletions with actor + timestamp
Settings	Manage AI model, toggle features (Google Vision fallback, push notifications)

12. Technical Notes & Annotations

Technical Notes

Implementation details, known limitations, and recommendations

Flutter OCR Notes

- ML Kit OCR (google_ml_kit ^0.16) supports Latin, Devanagari, Chinese, Japanese, Korean scripts — adequate for Philippine use (English/Filipino text).
- For handwritten text, recognition accuracy drops significantly. Recommend informing users to use printed text mode and adding a 'poor result?' feedback button.
- Image pre-processing: grayscale conversion and contrast enhancement via image package improves OCR accuracy by ~15–20% on low-quality captures.
- flutter_tts Filipino support depends on device TTS engine. Google TTS (Android) supports Filipino; iOS uses Siri TTS which may not. Test on target device.

Backend Notes

- Render.com free tier instances spin down after 15 minutes of inactivity — first request after sleep takes ~30 seconds. Use a cron ping (UptimeRobot free) to prevent cold starts.
- OpenAI API key must never be exposed in Flutter source. Always proxy through Express /api/ai/explain — never call OpenAI directly from the mobile client.
- Google Vision API requires a billing account even for free tier (1,000 req/mo). Set a hard budget cap of \$0 in GCP Console to prevent unexpected charges.
- Mongoose connection should use connection pooling (maxPoolSize: 10) and handle reconnection events — critical for Render.com's ephemeral instances.

MongoDB Notes

- M0 free cluster has a 512MB storage limit and 100 max connections. For capstone demo this is more than sufficient; upgrade to M10 (\$57/mo) only for production.
- Add TTL index on scan_history (expireAfterSeconds: 7776000) to auto-delete scans older than 90 days — keeps storage within M0 limits.
- Use Mongoose lean() queries in admin panel endpoints to reduce memory overhead when fetching large lists.

Security Notes

- JWT secret must be a cryptographically random 256-bit string — never use short strings like 'secret' or 'trisha'. Generate with: `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`
- Cloudinary upload preset must be set to 'signed' — unsigned presets allow anyone to upload to your account.
- CORS origin must explicitly list allowed domains in production — never use origin: '*' in the Express CORS config for a deployed API.
- All admin routes must double-check the role from the JWT payload server-side — never trust role information sent from the client.

Known Limitations

- ML Kit OCR does not support real-time/live camera OCR — requires image capture first. Live OCR (like Google Lens) requires a different SDK approach.
- Filipino TTS quality varies by Android device and TTS engine version. Recommend testing on Samsung and Xiaomi devices common in the Philippines.
- AI explanation latency (~1–3 seconds) may feel slow for accessibility users. Consider streaming responses (SSE) in a future version.
- The free Render.com tier does not support persistent disk storage — all file uploads must go directly to Cloudinary, not local disk.

This document constitutes the complete system architecture and project planning reference for the TRISHA capstone project. All architectural decisions are designed to maximize free-tier usage, minimize operational complexity during the capstone phase, and provide a clear upgrade path to production deployment. The document should be submitted alongside the Chapter 2 (Review of Related Literature) and presented during the technical review panel.

Document prepared for National University Philippines BSIT Capstone Program. Version 1.0.